LOGO

AI Task and Prompt Pack

Paste-ready prompts P01 to P33 that make an AI coding assistant build the project defined in the Execution Playbook, one small verified task at a time

Companion to	LOGO_Execution_Playbook.pdf (plan, risks, experiment design, paper blueprint)
Use with	Any capable AI coding assistant. Best results with a tool that can read and edit files inside your repo and run tests; a normal chat also works if you paste files as each task asks.
Also provided	LOGO_AI_Prompt_Pack.md: same text, copy-paste friendly (PDF line wraps can add line breaks).
Date	21 October 2026

1. How this pack works

The playbook says *what* to build and *why*. This pack is the *how to instruct the AI*. If every task is given with the same shared context and the same file contracts, separate chats produce modules that fit together.

Step	What you do	Why
1	Start a **new chat** for each task.	Long chats resend history and burn credits; fresh chats stay focused.
2	Paste **Block A** (Section 2), then your current **PROJECT_STATE.md**.	Gives the AI project rules, the frozen file/function contract and what is already built.
3	Paste the task's **PROMPT** plus the items listed under *You provide* (files, samples, errors).	The AI must see the real formats and third-party APIs; otherwise it guesses.
4	Run the commands under *You run*. Check the *Accept when* line.	Nothing is Done until it runs and the check passes.
5	If it fails, use a **recovery prompt** (Section 3); do not start over.	Cheapest path to a fix.
6	Commit, then ask for the PROJECT_STATE update (already part of every response) and paste it into the file.	The next chat inherits the truth about the repo.

What an AI can and cannot do here

- **It can** write the modules, tests, configs and scripts in every prompt, and debug from traces you paste.
- **It cannot** download datasets, install drivers, run GPU training on your laptop, look at your videos, or know the real format of files it has not seen. Those steps are yours: marked under *You provide* and *You run*.
- **It may be wrong.** Prompts demand tests and acceptance checks so mistakes show up early. Treat the P06 evaluator as the one module you review line by line, because every result depends on it.
- **Results are not guaranteed.** Following the pack gives you a working, honest pipeline; whether the improvements C2 and C3 help is decided by your ablations.

Execution order and gates

Run P01 to P33 in order (P07B only if Plan B, P21F only if P21 is blocked). A gate means: stop and verify before continuing.

Task	Plan card	Title	Env	Needs
P01	T1.1	Repo skeleton and shared utilities	logo-torch	none
P02	T1.2	Environment check and setup notes	logo-torch	P01
P03	T1.3	Dataset inventory	logo-torch	P01
P04	T1.4	Unified annotation schema and parsers	logo-torch	P03
P05	T1.5 + T1.6	Frames, GT visualizer, statistics, splits	logo-torch	P04
P06	T2.1	Evaluation harness (detection, tracking, spotting)	logo-torch	P04
P07A	T2.2	Detector inference to det.json (Plan A: PP-YOLOE-R, Paddle)	logo-paddle	P02, P04
P07B	T2.2 (Plan B only)	Detector inference to det.json (Plan B: YOLO OBB, PyTorch)	logo-torch	P02, P04
P08	T2.3 to T2.5	Detector training data, fine-tune and export (Plan A)	logo-paddle	P07A
P09	T3.1	RoI-Rotate crop utility	logo-torch	P04
P10	T3.2	ABINet wrapper and baseline recognition accuracy	logo-torch	P09
P11	T3.3	LSC dataset builder (paper Eq. 1 to 3)	logo-torch	P08 (or P07B), P09
P12	T3.4	Fine-tune the LSC	logo-torch	P10, P11

Task	Plan card	Title	Env	Needs
P13	T3.5	LSC inference, hard fusion, detection check	logo-torch	P12
P14	T4.1	Glyph pseudo-labels (K-means)	logo-torch	P11
P15	T4.2	Glyph branch with MSE loss (time-boxed)	logo-torch	P12, P14
P16	T4.3	Baseline tracker (ByteTrack-style)	logo-torch	P13
P17	T4.4	Pipeline runner and first ablation rows	logo-torch	P13, P16, P06
P18	T4.5	VPMM module	logo-torch	P01
P19	T5.1	GoMatching LST-Matcher study and adapter plan	logo-torch	P18
P20	T5.2	Feature cache for LST-Matcher training	logo-torch	P13, P18, P19
P21	T5.3	Train the LST-Matcher	logo-torch	P20
P21F	T5.3 fallback	Fallback tracker: ByteTrack-style + VPMM embedding	logo-torch	P18, P16
P22	T5.4	LST inference, thresholds, trajectory filtering	logo-torch	P21 (or P21F)
P23	T5.5	Baseline freeze and efficiency profile	logo-torch	P22
P24	T6.1	Error analysis on the frozen baseline	logo-torch	P23
P25	T6.2	C2: soft confidence-aware language re-scoring	logo-torch	P24
P26	T6.3	C3: track-level text voting and filtering	logo-torch	P25
P27	T6.4 + T6.5	Ablation runner and bootstrap confidence intervals	logo-torch	P26
P28	T7.1 + T7.2	Final runs and efficiency table	logo-torch	P27
P29	T7.3 + T7.5	Tables and figures generated by script	logo-torch	P28
P30	T7.4	Demo	logo-torch	P28
P31	T8.3	README and reproducibility guide	any	P29
P32	T8.4	IEEE paper skeleton and first draft	Overleaf / any	P29
P33	T8.1 + T8.2	Project report and final presentation drafts	any	P29, P32

When	Gate	Must be true
After P05	Data gate	Datasets parsed, frames and overlays verified, splits fixed (M1, M2).
After P08	Detector gate	Detection F recorded, det.json exists for all videos (M3).
After P13	LSC gate	Precision improves after re-scoring (M4).
After P17	First-table gate	Baseline vs Baseline+LSC reproduced.
After P23	MVP gate	Frozen baseline reproduces within 0.3; ready to submit even if nothing else finishes.
After P27	Claims gate	Ablation + confidence intervals decide which claims you may write.
After P28	Freeze gate	Final numbers stored; stop changing code.

2. Block A: paste at the start of EVERY task chat

Block A holds the project description, working rules, response format and the frozen CONTRACT (paths, JSON formats, function signatures, config keys). Do not edit it casually: if a contract change is truly needed, change it here and in every module that uses it, and record it in PROJECT_STATE.md.

```
PROJECT: B.Tech major project "LOGO: video text spotting" = re-implementation of arXiv 2405.19194 (detector
-> language synergy classifier (LSC, ABINet-based) -> score fusion -> VPMM features -> LST-Matcher tracker)
plus two extensions: C2 soft language re-scoring and C3 track-level text voting.
PIPELINE (each stage reads the previous stage's files from disk): frames -> det -> lsc -> fuse -> track ->
vote -> eval.
ENV: Python 3.11, PyTorch 2.1.2+cu121, NumPy 1.26.4, one RTX 3050 6GB laptop GPU (use AMP, small batches,
never load a whole dataset into RAM). Only the detector may run in a separate PaddlePaddle env
(logo-paddle). Code must run on Windows/WSL2/Linux (pathlib, no shell-only logic).
RULES:
1) Follow the CONTRACT below exactly. Never rename files, JSON keys, config keys or function signatures. If
a change is truly needed, say so first and wait for my answer.
2) Write complete runnable code: no stubs, no TODO placeholders, no invented third-party APIs. If unsure
about a library or repo API, tell me which file to paste instead of guessing.
3) Every new module gets a pytest in tests/ that runs on CPU with synthetic data in under 30 s.
4) Seeds come from the config. Every run writes runs/<exp_id>/{config.yaml,log.txt,metrics.json}.
5) Never use the test split unless the script is called with --final.
6) Keep answers short: no tutorials, no restating my prompt.
RESPONSE FORMAT: (1) files created/changed; (2) full code for each new file, or a unified diff for edits;
(3) tests; (4) exact commands to run; (5) five lines I can paste into PROJECT_STATE.md; (6)
assumptions/limits in at most 3 lines.
CONTRACT
Paths: data/raw/<ds>/ ; data/frames/<ds>/<video>/%06d.jpg (frame ids 1-based) ;
data/processed/<ds>/<video>.json (GT) ; splits/<ds>_{train,val,test}.txt (one video id per line) ;
runs/<exp_id>/<stage>/<ds>/<video>.<ext>. ds in {icdar2013v, icdar2015v, dstext}.
GT json: {"video","dataset","width","height","num_frames","frames":{"1":[{"track_id":int,"quad":[x1,y1,x2,y
2,x3,y3,x4,y4],"text":str,"dont_care":bool,"quality":str|null}]}}
det json: {"video","dataset","frames":{"1":[{"quad":[8 floats],"score":float}]}}
lsc json: det objects plus {"text":str,"text_conf":float,"p_eos":float,"lang_hard":0|1,"feat_idx":int};
features in <video>_feats.npy float16 [N,c,d], row = feat_idx (counted across the video in file order).
fused json: lsc objects plus {"score_fused":float}.
tracks: <video>.jsonl, one line per box: {"frame":int,"track_id":int,"quad":[8],"score":float,"text":str}
metrics.json: {"det":{"precision","recall","f"}, "track":{"MOTA","MOTP","IDF1","MT","ML"}, "spot":{same
keys}, "per_video":{video:{"frames","num_objects","misses","fp","switches","idtp","idfp","idfn"}},
"meta":{}}. All metric values in percent; MOTP is IoU-based (higher is better).
Package: src/logo/{utils,data,detector,lsc,glyph,vpmm,tracking,scoring,voting,eval}; import as
logo.<pkg>.<mod>.
Functions: logo.utils.io: load_json(p), save_json(o,p), load_yaml(p), set_seed(s), get_logger(name,logfile)
logo.data.schema: load_gt(path)->dict ; list_videos(ds,split)->list[str]
logo.eval.det_eval.evaluate_detection(gt_dir,pred_dir,videos,iou=0.5)->dict
logo.eval.mot_eval.evaluate_mot(gt_dir,track_dir,videos,spotting,iou=0.5)->dict
logo.lsc.roi_rotate.crop_quad(img,quad,out_hw=(32,128))->np.ndarray uint8 HxWx3
logo.scoring.fusion.fuse(det,lang,mode,w=0.5)->float (mode "hard" or "soft")
logo.tracking.simple_tracker.run_tracker(fused,cfg)->list[dict]
logo.vpmm.module.VPMM(c,d): forward(quads[N,4,2], feats[N,c,d], img_wh[N,2]) -> [N,2d]
logo.voting.track_vote.vote_tracks(tracks,cfg)->list[dict]
Config keys (yaml): exp_id, dataset, split, seed, detector{}, lsc{ckpt,det_thr:0.3,iou_neg:0.05},
fusion{mode,w}, tracker{type:simple|lst,thr}, voting{enabled,tau,min_len}, eval{iou:0.5,spotting:true}
```

3. PROJECT_STATE.md template and recovery prompts

Keep this file under 40 lines in the repo root. Paste it after Block A in every chat. After each task, add the five lines the AI gives you.

```
# PROJECT_STATE.md (keep under 40 lines)
## Done (task ids + one line each)
P01 repo skeleton, io utils
## Decisions / deviations from the paper
- (example) detector: Plan A PP-YOLOE-R small, input 1024x576
## Interfaces added beyond the CONTRACT (name, signature, file)
-
## Open problems
-
## Next task
P02
```

Recovery prompts

Fix an error (cheapest)

```
TASK: fix this error only. Do not refactor. Rules and CONTRACT from Block A still apply.
FILE (only the relevant function):
<paste function>
ERROR (last 30 lines):
<paste trace>
Give a minimal unified diff and the command to re-run.
```

Tests pass but numbers look wrong

```
TASK: diagnose, do not rewrite. Expected: <what you expected>. Got: <metrics>. Give me a ranked list of at most 5 likely causes, and for each one a 5-line check I can run (print/assert) to confirm or exclude it. Only then propose a fix for the confirmed cause.
```

AI broke the CONTRACT

```
You changed <name> which Block A forbids. Revert to the CONTRACT signature/keys and adapt the new code to it. Output only the diff.
```

Resume in a fresh chat

```
<paste Block A> <paste PROJECT_STATE.md> TASK: continue with <task id>. Before coding, restate in 3 lines which files from earlier tasks you will import, and list any you need me to paste.
```

Credit-saving reminders

- Paste only the function and the last 30 lines of an error, never whole files unless the task asks.
- Run the smoke mode (`--smoke`) before long runs; only then ask for fixes.
- If the AI proposes a redesign, answer 'keep the CONTRACT, minimal diff'.
- When a chat gets long (about 15 messages) restart with the 'Resume in a fresh chat' prompt.

4. Task prompts P01 to P33

Each block: dependencies, what you must paste, the prompt, commands, and the acceptance test. Text in angle brackets such as <paste> is a placeholder you replace.

Sprint 1: Foundation

P01 Repo skeleton and shared utilities

Plan: T1.1 | Env: logo-torch

Depends on: none

You provide (paste with the prompt): Nothing (Block A only).

PROMPT (paste after Block A):

```
TASK P01. Create the repository skeleton exactly as in the CONTRACT:
- src layout with every package under src/logo (utils, data, detector, lsc, glyph, vpmm, tracking, scoring, voting, eval), each with __init__.py; pyproject.toml so that `pip install -e .` installs package `logo`.
- .gitignore (data/, runs/, *.ckpt, *.pth, *.pdparams, __pycache__, .venv, LaTeX build files), README.md skeleton, PROJECT_STATE.md (from my template), docs/decisions.md, configs/base.yaml containing every config key from the CONTRACT with sensible defaults.
- Implement logo.utils.io with the five functions (logger writes to console and to the given file; set_seed seeds python, numpy, torch).
- tests/test_io.py covering all five functions.
Do not add any other modules.
```

You run:

```
$ pip install -e .
$ pytest -q
```

✔ **Accept when:** pytest passes; `python -c "import logo"` works; repo pushed to GitHub.

P02 Environment check and setup notes

Plan: T1.2 | Env: logo-torch (+ logo-paddle notes)

Depends on: P01

You provide (paste with the prompt): Your OS (Windows or WSL2), NVIDIA driver version (nvidia-smi), and the output of your current torch check.

PROMPT (paste after Block A):

```
TASK P02. (a) Write tools/check_env.py: print Python, torch, CUDA availability, GPU name, total VRAM, numpy,
opencv, shapely, motmetrics, sklearn, yaml versions; exit non-zero if a required package is missing or CUDA
is unavailable.
(b) Write docs/env_setup.md with exact commands for: the logo-torch env (already Python 3.11, torch
2.1.2+cu121; add opencv-python, shapely, pyclipper, scikit-learn, pyyaml, tqdm, pytest, motmetrics,
matplotlib); and a SEPARATE logo-paddle env for PaddleDetection (Python 3.9 or 3.10; install
PaddlePaddle-GPU using the official install selector for my driver; clone PaddleDetection and pin the
commit; build the rotated-IoU custom op; verify with paddle.utils.run_check()). Do not invent Paddle version
numbers: write 'choose from the official selector'.
(c) Add a 'Plan B' section: a PyTorch rotated detector (Ultralytics YOLO OBB) inside logo-torch, with what
changes in the pipeline (nothing downstream).
My OS/driver: <paste>
```

You run:

```
$ python tools/check_env.py
$ # then follow docs/env_setup.md for logo-paddle (time-box 1 day)
```

✔ **Accept when:** check_env passes in logo-torch AND (paddle run_check passes OR Plan B recorded in docs/decisions.md).

P03 Dataset inventory

Plan: T1.3 | Env: logo-torch

Depends on: P01

You provide (paste with the prompt): Output of a directory listing (2 levels) of data/raw/icdar2013v, icdar2015v and dstext (Windows: tree /F, depth limited, or ls -R | head -60).

PROMPT (paste after Block A):

```
TASK P03. Write tools/inventory.py --root data/raw. For each dataset folder (icdar2013v, icdar2015v,
dstext): count videos and annotation files, match them by file stem, list videos without annotations and
annotations without videos, and print a table. Append the result under '## Inventory' in docs/data_report.md
(create if missing). Base the matching on the real naming below; if the naming is ambiguous, say what you
assumed.
Directory listings: <paste>
Expected counts (from papers): ICDAR2013 Video 28 videos; ICDAR2015 Video 49 videos (25 train, 24 test);
DSText 100 videos (50 train, 50 test).
```

You run:

```
$ python tools/inventory.py --root data/raw
```

✔ **Accept when:** counts match expectations or every difference is explained in docs/data_report.md.

P04 Unified annotation schema and parsers

Plan: T1.4 | Env: logo-torch

Depends on: P03

You provide (paste with the prompt): First 40 lines of ONE annotation file per dataset (three samples). Without them the AI will guess the format; do not skip.

PROMPT (paste after Block A):

```
TASK P04. Write logo.data.parsers (one parser per dataset: parse_icdar2013v, parse_icdar2015v, parse_dstext)
and logo.data.schema (load_gt, list_videos) plus tools/convert_annotations.py that writes
data/processed/<ds>/<video>.json in the GT schema of the CONTRACT.
Read the annotation samples below and write parsers for exactly those formats. If a sample is missing, stop
and ask for it. Rules: transcription '###' means dont_care=true; keep 'quality' if present, else null; frame
ids 1-based; track_id from the annotation ID (unique per video); width/height from the first frame image if
the file lacks them (read data/frames if present, else leave 0 and flag it).
Tests: use tiny synthetic annotation snippets written to a temp dir, assert quad length 8, frame ids
contiguous or listed, IDs stable. After conversion print a summary table per dataset (videos, frames,
instances, dont_care count).
Samples: <paste three samples>
```

You run:

```
$ python tools/convert_annotations.py --root data/raw --out data/processed
$ pytest -q tests/test_parsers.py
```

✔ **Accept when:** all three datasets convert with no error; summary printed and saved to docs/data_report.md. Also confirm the DONT-CARE rule in each dataset's evaluation readme and record it.

P05 Frames, GT visualizer, statistics, splits

Plan: T1.5 + T1.6 | Env: logo-torch

Depends on: P04

You provide (paste with the prompt): Nothing, but have a few videos ready.

PROMPT (paste after Block A):

```
TASK P05. Write: (1) tools/extract_frames.py (OpenCV, JPEG quality 90, names %06d.jpg starting at 000001, optional --stride, skips already extracted videos); (2) tools/viz_gt.py --ds --video --out that draws quads, track ids and text on frames and writes an mp4; (3) tools/dataset_stats.py computing videos, frames, instances, tracks, dont_care share, text height histogram and the share of small text (quad height below 20 px), saved as CSV and two PNG figures into docs/figs and summarised into docs/data_report.md; (4) tools/make_splits.py: for each dataset use the official train videos, create train/val by video (80/20, seed from config), write splits/<ds>_{train,val,test}.txt (test = official test videos; if test annotations are missing write them anyway and print a warning that val acts as held-out test); assert no video appears in two splits. Tests for the split logic.
```

You run:

$ `python tools/extract_frames.py --ds icdar2015v`

$ `python tools/viz_gt.py --ds icdar2015v --video <id> --out runs/viz.mp4`

$ `python tools/dataset_stats.py`

$ `python tools/make_splits.py`

✔ **Accept when:** frame count equals annotated range for 3 videos per dataset; you watched the overlay videos and boxes line up; splits committed (milestone M2).

Sprint 2: Evaluation and detector

P06 Evaluation harness (detection, tracking, spotting)

Plan: T2.1 | Env: logo-torch

Depends on: P04

You provide (paste with the prompt): Nothing (py-motmetrics is installed). This is the most important module: review it carefully.

PROMPT (paste after Block A):

```
TASK P06. Implement logo.eval.det_eval and logo.eval.mot_eval plus tools/evaluate.py --gt_dir --pred_dir --videos splits/<ds>_val.txt --mode det|track|spot --out metrics.json.
Details: polygon IoU with shapely (cache polygons; prefilter by axis-aligned box overlap). Default iou=0.5.
DONT-CARE handling: a prediction is ignored (neither TP nor FP) if more than 50% of its area lies inside a dont_care GT quad of the same frame; dont_care GT never counts as a miss.
det mode: precision, recall, f (percent) from greedy one-to-one matching by IoU, per frame.
track mode: py-motmetrics accumulator with distance 1-IoU, pairs with IoU below the threshold set to NaN. Report MOTA, MOTP (as IoU = 1 - motp), IDF1, MT (GT tracks covered at least 80% of lifetime), ML (covered at most 20%).
spot mode: same, but a pair is only allowed when normalized texts are equal (lowercase, keep only [a-z0-9]); GT with empty normalized text is dont_care.
Also write per-video counts into metrics.json["per_video"] exactly as in the CONTRACT (frames, num_objects, misses, fp, switches, idtp, idfp, idfn) so a bootstrap over videos is possible later.
tests/test_eval.py (synthetic): GT used as prediction gives 100 on every metric; boxes shifted by half a box width reduce MOTA; swapping two track ids mid-video lowers IDF1 but not detection f; shuffling texts lowers only spot mode; a prediction inside a dont_care region is ignored.
```

You run:

$ `pytest -q tests/test_eval.py`

$ `python tools/evaluate.py --gt_dir data/processed/icdar2015v --pred_dir <gt-as-pred dir> --videos splits/icdar2015v_val.txt --mode spot --out /tmp/m.json`

✔ **Accept when:** all five sanity tests pass; running on GT-as-prediction for one real video returns 100. If the official challenge scripts are available, compare on one video and note differences in docs/decisions.md.

P07A Detector inference to det.json (Plan A: PP-YOLOE-R, Paddle)

Plan: T2.2 | Env: logo-paddle

Depends on: P02, P04

You provide (paste with the prompt): Paste PaddleDetection tools/infer.py (or deploy/python/infer.py) and the rotate config you plan to use (configs/rotate/ppyoloe_r/...). The AI must use the API in those files.

PROMPT (paste after Block A):

```
TASK P07A. Write src/logo/detector/infer_paddle.py: load a PP-YOLOE-R (small) checkpoint using the same API as the pasted infer.py, run on all frames of the videos in splits/<ds>_<split>.txt (batch size from config), and write runs/<exp_id>/det/<ds>/<video>.json in the det schema of the CONTRACT (quad = 8 floats in image coordinates, score; keep scores above detector.save_thr=0.1). Print peak GPU memory and ms/frame at the end and save them into runs/<exp_id>/det/timing.json. Use AMP only if the pasted code supports it. This script may import ONLY paddle, ppdet, numpy, cv2 and logo.utils.io (it runs in the paddle env).
```

```
Pasted files: <infer.py> <config>
```

You run:

$ `python -m logo.detector.infer_paddle --config configs/E2-det.yaml --split val --max_videos 1`

✔ **Accept when:** det.json written for one video; a drawn overlay (use tools/viz_det.py if you ask for it) shows boxes on text; timing.json present.

P07B Detector inference to det.json (Plan B: YOLO OBB, PyTorch)

Plan: T2.2 (Plan B only) | Env: logo-torch

Depends on: P02, P04

You provide (paste with the prompt): Use this ONLY if P02 decided Plan B. Say which Ultralytics version you installed (pip show ultralytics).

PROMPT (paste after Block A):

```
TASK P07B (Plan B). Write src/logo/detector/infer_obb.py using the Ultralytics oriented-box (OBB) model API
to run on all frames of the videos in splits/<ds>_<split>.txt and write det.json per video exactly as in the
CONTRACT (8-float quads, scores above detector.save_thr=0.1). Also write
src/logo/detector/prepare_obb_data.py converting data/processed GT (non-dont_care) into the label format
Ultralytics OBB expects (one class 'text', normalized corner coordinates) and a dataset yaml, using frame
stride from config. Record peak GPU memory and ms/frame in timing.json. State the exact Ultralytics calls
you rely on; if unsure of a call, ask me to run help() and paste.
```

You run:

$ `python -m logo.detector.prepare_obb_data --config configs/E2-det.yaml`

$ `python -m logo.detector.infer_obb --config configs/E2-det.yaml --split val --max_videos 1`

✔ **Accept when:** same as P07A; deviation recorded in docs/decisions.md.

P08 Detector training data, fine-tune and export (Plan A)

Plan: T2.3 to T2.5 | Env: logo-paddle (training may run on Kaggle/Colab or a lab GPU)

Depends on: P07A

You provide (paste with the prompt): The training config you will modify and the dataset-format section of the PaddleDetection rotate docs (paste the relevant lines).

PROMPT (paste after Block A):

```
TASK P08. (1) Write tools/gt_to_dota.py converting data/processed GT (non-dont_care) of train videos to the
polygon format the pasted docs require (DOTA-style: 8 coordinates, category 'text', difficult flag; set
dont_care regions to difficult=1 or omit, say which), using --stride 2 for frames, plus the dataset file
listing. (2) Produce configs/ppyoloe_r_text_s.yml derived from the pasted config: small model, input about
1024x576, batch size 2, AMP on, SGD momentum 0.9, weight decay 5e-4, warm-up then cosine, checkpoint every
epoch, resumable. Show only the changed keys. (3) Write scripts/train_detector.sh with resume support and a
smoke mode (200 images, 1 epoch). (4) Write tools/export_all_dets.py that runs infer_paddle over
train/val/test with save_thr 0.1.
Pasted: <train config> <dataset docs lines>
```

You run:

$ `bash scripts/train_detector.sh --smoke`

$ `bash scripts/train_detector.sh`

$ `python tools/export_all_dets.py --exp E2-det-v1`

$ `python tools/evaluate.py --mode det ... (validation F)`

✔ **Accept when:** smoke run finishes without OOM; validation detection F recorded; det.json exists for every video (milestone M3).

Sprint 3: Language synergy classifier

P09 RoI-Rotate crop utility

Plan: T3.1 | Env: logo-torch

Depends on: P04

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P09. Implement logo.lsc.roi_rotate.crop_quad(img, quad, out_hw=(32,128)) and crop_many(img, quads).
Steps: order the four vertices consistently (top-left, top-right, bottom-right, bottom-left) from arbitrary
input order and orientation; perspective-warp to out_hw with cv2.getPerspectiveTransform/warpPerspective; if
the source quad is clearly vertical (height more than 1.5 times width) rotate the crop 90 degrees so text
runs left to right; return uint8 HxWx3. Add tools/crop_grid.py saving a grid PNG of 64 crops from GT or
det.json for a video. tests/test_roi.py on a synthetic image with a rendered rotated word: crop height/width
equal out_hw and the crop of an axis-aligned quad equals the plain resize.
```

You run:

```
$ pytest -q tests/test_roi.py
$ python tools/crop_grid.py --ds icdar2015v --video <id> --out runs/crops.png
```

✔ **Accept when:** tests pass; the crop grid shows readable, upright text for oriented words.

P10 ABINet wrapper and baseline recognition accuracy

Plan: T3.2 | Env: logo-torch

Depends on: P09

You provide (paste with the prompt): Clone the official PyTorch ABINet into third_party/ABINet and download its pretrained weights. Paste demo.py, the model config yaml and the CharsetMapper class (from utils.py).

PROMPT (paste after Block A):

```
TASK P10. Write logo.lsc.abinet_wrapper.ABINetRecognizer with: __init__(ckpt, device, fp16=True);
predict(crops: list[np.ndarray]) -> list of dicts {text, text_conf, p_eos, feat} where text is lowercase
decoded string, text_conf the mean character probability, p_eos the probability of the end/null token at
decoding step 0 from the final (fused) head (index 0 of the repo's CharsetMapper), and feat the per-instance
visual feature [c,d] from the vision model attention output (read the true c and d from the config and tell
me). Reuse demo.py's model loading; import ABINet code from third_party via a documented sys.path insert in
one place only. Then tools/eval_recognition.py: crops of GT (non-dont_care) in the val split, word accuracy
(case-insensitive alphanumeric) and a CSV of predictions.
Pasted: <demo.py> <config> <CharsetMapper>
```

You run:

```
$ python tools/eval_recognition.py --ds icdar2015v --split val --ckpt <abinet pretrained>
```

✔ **Accept when:** word accuracy number recorded in docs/baseline_report.md; batch inference runs inside 6 GB; c and d recorded in PROJECT_STATE.md.

P11 LSC dataset builder (paper Eq. 1 to 3)

Plan: T3.3 | Env: logo-torch

Depends on: P08 (or P07B), P09

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P11. Write tools/build_lsc_dataset.py. Positives: crops (crop_quad) of GT instances that are not
dont_care, label = lowercase text restricted to the ABINet charset (drop other characters; drop the instance
if nothing remains). Negatives: detector outputs (runs/<exp>/det) with score above lsc.det_thr=0.3 whose IoU
with ALL GT quads of that frame (including dont_care) is below lsc.iou_neg=0.05, label = empty string (so
the first token is the end token). Balance positives:negatives = 1:1 per split with the seed (cap by the
smaller), use frame stride from config, write PNG crops under data/lsc/<split>/images and
data/lsc/<split>/labels.tsv (relative path TAB label) plus stats.json (counts, text length histogram). Add
logo.lsc.dataset.LSCDataset (PyTorch Dataset) returning image tensor and label string with training
augmentation switch (rotation, affine, perspective, blur/quality loss, color jitter). Tests: IoU filter
keeps a far box and rejects an overlapping one.
```

You run:

```
$ python tools/build_lsc_dataset.py --exp E2-det-v1 --split train
$ python tools/build_lsc_dataset.py --exp E2-det-v1 --split val
```

✔ **Accept when:** stats.json saved; you opened 50 random negatives and most are background or text-like, and no negative lies on obvious text.

P12 Fine-tune the LSC

Plan: T3.4 | Env: logo-torch

Depends on: P10, P11

You provide (paste with the prompt): Paste the ABINet repo's training entry (main.py / Learner, loss construction) and the fine-tuning yaml.

PROMPT (paste after Block A):

```
TASK P12. Write src/logo/lsc/train_lsc.py (and configs/E3-lsc.yaml) to fine-tune the pretrained ABINet on
LSCDataset: reuse the repo's own losses (vision, language and fusion cross-entropy; iterative refinement)
and CharsetMapper; empty-string label must encode as end token at step 0 then padding. Settings: Adam lr
1e-4, decay to 1e-5 after epoch 6, up to 10 epochs, batch 48 (reduce if OOM), AMP, gradient accumulation
option, checkpoint per epoch plus best by validation text/background accuracy, early smoke mode (--smoke:
200 samples, 1 epoch). Validation reports: text/background accuracy (first decoded token is end token or
not) and word accuracy on positives; write to runs/<exp>/metrics.json under meta. Keep the repo code
unmodified; subclass or wrap instead.
Pasted: <training entry> <loss code> <yaml>
```

You run:

```
$ python -m logo.lsc.train_lsc --config configs/E3-lsc.yaml --smoke
$ python -m logo.lsc.train_lsc --config configs/E3-lsc.yaml
```

✔ **Accept when:** validation text/background accuracy and word accuracy recorded in runs/E3-lsc-v1; best checkpoint saved.

P13 LSC inference, hard fusion, detection check

Plan: T3.5 | Env: logo-torch

Depends on: P12

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P13. (1) logo.lsc.run_lsc: for each video, read det.json, keep boxes above lsc.det_thr, crop with
crop_quad, run ABINetRecognizer in batches, write lsc json plus <video>_feats.npy (float16, row = feat_idx)
per the CONTRACT; lang_hard = 1 if the first decoded token is not the end token else 0. (2)
logo.scoring.fusion.fuse(det, lang, mode, w): mode 'hard' -> (1-w)*det + w*lang_hard (w=0.5 equals the
paper's average); mode 'soft' raises NotImplementedError until task P25. (3) logo.scoring.run_fuse writing
fused json. (4) tools/rescore_panel.py: before/after overlay of the same frame coloring boxes by score (blue
low to red high). (5) Evaluate detection P/R/F on val before (det score above 0.3) vs after (fused score
above 0.3 and above 0.5) using tools/evaluate.py; write a small table to docs/baseline_report.md.
```

You run:

```
$ python -m logo.lsc.run_lsc --config configs/E3-lsc.yaml
$ python -m logo.scoring.run_fuse --config configs/E3-lsc.yaml
$ python tools/evaluate.py --mode det ...
```

✔ **Accept when:** precision rises after re-scoring (paper trend); rescore panel PNG saved; tag v0.3-detector.

Sprint 4: Glyph, tracker, VPMM

P14 Glyph pseudo-labels (K-means)

Plan: T4.1 | Env: logo-torch

Depends on: P11

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P14. Write logo.glyph.pseudo_labels.kmeans_mask(crop_rgb) -> float mask [H,W] in {0,1}: convert to Lab,
run k-means with k=2 on pixels (fixed seed, small n_init), choose the text cluster by the rule 'the cluster
with fewer pixels on the crop border ring', optional 3x3 morphological cleanup. Then
tools/make_glyph_masks.py generating masks for all LSC positive training crops into data/lsc/<split>/masks/.
Provide tools/mask_grid.py to visualize 50 crop/mask pairs side by side. Test on a synthetic crop with a
bright word on a dark background (mask overlaps the word with IoU above 0.7) and on the inverted colours
(same).
```

You run:

```
$ python tools/make_glyph_masks.py --split train
$ python tools/mask_grid.py --split train --out runs/masks.png
```

✔ **Accept when:** masks exist for all positive training crops; the grid shows mostly plausible masks, failures noted.

P15 Glyph branch with MSE loss (time-boxed)

Plan: T4.2 | Env: logo-torch

Depends on: P12, P14

You provide (paste with the prompt): Paste the ABINet vision backbone (ResNet) class so feature taps can be added, and the shapes of its three feature levels.

PROMPT (paste after Block A):

```
TASK P15. Add logo.glyph.branch.GlyphHead per paper Eq. 5: inputs backbone features F0,F1,F2 (coarse to fine as in the pasted backbone; tell me their channels/strides); G2 = phi(F2); G1 = phi(up2(G2) + F1); Sm = tau(up2(G1) + F0) where phi = conv3x3+BatchNorm+ReLU and tau = three conv layers with ReLU ending in a 1-channel sigmoid map. Use 1x1 convs to match channels if needed (state this deviation). Loss L_seg = MSE(Sm, S_pl) with pseudo-masks resized to Sm. Extend train_lsc.py with flag glyph.enabled and total loss L_lsc = L_rec + L_seg (weight in config, default 1). Wrap the backbone with forward hooks instead of editing repo code. Output: ablation script tools/compare_glyph.py reporting word accuracy with vs without glyph on the full val set and on a noisy subset (the 30% of val crops with lowest mean image gradient magnitude as a blur proxy). Optionally a BCE-vs-MSE flag. Time-box: if results are within noise, report that.
```

You run:

```
$ python -m logo.lsc.train_lsc --config configs/E4-lsc-glyph.yaml --smoke
$ python tools/compare_glyph.py --base E3-lsc-v1 --glyph E4-lsc-glyph-v1
```

✔ **Accept when:** comparison table saved; keep the better checkpoint as lsc.ckpt for later tasks and record the decision.

P16 Baseline tracker (ByteTrack-style)

Plan: T4.3 | Env: logo-torch

Depends on: P13

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P16. Implement logo.tracking.simple_tracker.run_tracker(fused, cfg) and a CLI logo.tracking.run_track. Two-stage association as in ByteTrack: detections with score_fused above tracker.high_thr (default 0.6) are matched first to existing tracks by quad IoU (Hungarian via scipy, IoU gate 0.3), then remaining tracks are matched to detections between tracker.low_thr (default 0.3) and high_thr; unmatched high-score detections start new tracks; tracks die after tracker.max_age (default 15) frames; constant-velocity prediction of the quad centre. Track text = text of the highest-confidence recent detection. Output tracks jsonl per the CONTRACT (score = score_fused). Quad IoU via shapely with a bounding-box prefilter. Tests on synthetic moving boxes: ids persist across a 3-frame gap, two crossing boxes keep distinct ids.
```

You run:

```
$ pytest -q tests/test_tracker.py
$ python -m logo.tracking.run_track --config configs/E4-base.yaml
```

✔ **Accept when:** tracks.jsonl for all val videos; one overlay clip with ids watched (tools/viz_tracks.py if you ask for it).

P17 Pipeline runner and first ablation rows

Plan: T4.4 | Env: logo-torch

Depends on: P13, P16, P06

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P17. Write scripts/run_pipeline.py --config <yaml> --stage det,lsc,fuse,track,vote,eval [--force] [--final]. Each stage calls the existing module, reads/writes runs/<exp_id>/<stage>/<ds>/<video>.<ext>, and is skipped when outputs exist unless --force. det stage only checks that det files exist (detection runs in the paddle env). vote stage is a pass-through copy until task P26. eval stage runs det/track/spot evaluation on the split and writes runs/<exp_id>/metrics.json. Refuse split=test unless --final. Log timing per stage. Add configs/E4-base.yaml (no LSC: fusion uses det score only, i.e. w=0) and configs/E4-lsc.yaml (hard fusion w=0.5). Add tools/compare_runs.py printing a Markdown table of selected metrics for several exp ids.
```

You run:

```
$ python scripts/run_pipeline.py --config configs/E4-base.yaml --stage lsc,fuse,track,eval
$ python scripts/run_pipeline.py --config configs/E4-lsc.yaml --stage lsc,fuse,track,eval
$ python tools/compare_runs.py E4-base E4-lsc
```

✔ **Accept when:** one command regenerates both rows; Baseline+LSC beats Baseline on ICDAR2015 val (paper trend), or you record why not.

P18 VPMM module

Plan: T4.5 | Env: logo-torch

Depends on: P01

You provide (paste with the prompt): The c and d values from P10.

PROMPT (paste after Block A):

```
TASK P18. Implement logo.vpmm.module.VPMM(c, d) per paper Eq. 7 and 8, plus
logo.vpmm.geometry.centerline(quads, c, img_wh). Geometry: for each quad compute the four edge lengths; take
the two longest edges as the long sides, call the one with smaller mean y the top boundary, the other the
bottom; orient both left-to-right; sample c equally spaced points on each; centerline = mean of
corresponding points -> [N,c,2]; normalize x by image width and y by image height. Module: pos =
ReLU(Linear(2,d)) then ReLU(Linear(d,d)) on the centerline -> [N,c,d]; add the visual features feats
[N,c,d]; flatten to [N,c*d]; two Linear+ReLU layers c*d -> 2d -> 2d; return [N,2d]. forward(quads[N,4,2],
feats[N,c,d], img_wh) (img_wh tensor [N,2] or tuple). Tests: output shape for random input; centerline of an
axis-aligned rectangle lies on its horizontal midline; invariance to vertex order of the input quad (same
output).
```

You run:

```
$ pytest -q tests/test_vpmm.py
```

✔ **Accept when:** tests pass; module runs on a batch of cached LSC features from P13 without error.

Sprint 5: LST-Matcher and baseline freeze

P19 GoMatching LST-Matcher study and adapter plan

Plan: T5.1 | Env: logo-torch

Depends on: P18

You provide (paste with the prompt): Clone GoMatching into third_party/GoMatching. Paste the LST-Matcher module file(s), its config, and the loss/training step code (search the repo for 'LST', 'matcher', 'ST-Matcher', 'LT-Matcher').

PROMPT (paste after Block A):

```
TASK P19 (analysis, little code). Read the pasted GoMatching files and answer: (1) the exact input each
matcher expects per detection (tensor name, shape, dtype) and where it is built from DeepSolo queries; (2)
how a clip/batch is formed and how GT track labels enter the loss (paper Eq. 15 to 19); (3) the inference
call that returns associations; (4) which lines must change to feed our fused vector [N,2d] from VPMM
instead. Then write src/logo/tracking/lst_adapter.py: LSTAdapter wrapping the matcher so that
train_step(clip) and associate(frame_feats) take our arrays (quads, VPMM output, fused scores, track ids)
and hide GoMatching internals. Do not edit third_party files; wrap or subclass. If something cannot be
decided from the pasted code, list exactly what to paste next.
Pasted: <matcher files> <config> <loss/training code>
```

You run:

```
$ pytest -q tests/test_lst_adapter.py (synthetic tensors, CPU)
```

✔ **Accept when:** adapter plan committed to docs/decisions.md; adapter forward/backward runs on random tensors on CPU.

P20 Feature cache for LST-Matcher training

Plan: T5.2 | Env: logo-torch

Depends on: P13, P18, P19

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P20. Write tools/build_lst_cache.py: for each training video, read fused json (instances with
score_fused above 0.3), load feats.npy, compute VPMM inputs (centerline needs only quads and image size),
match every instance to GT by IoU above 0.5 (greedy, highest IoU, dont_care instances dropped) to obtain gt
track_id or -1, and save data/cache/<ds>/<video>.npz with arrays: frame_ids, quads [N,8], feats [N,c,d]
(float16), score_fused, track_ids, plus index offsets per frame. Write
logo.tracking.cache_dataset.LSTCacheDataset(clip_len=6, stride=1) returning clips of 6 consecutive frames
from one video (never crossing videos) as the adapter expects. Test with a tiny synthetic npz.
```

You run:

```
$ python tools/build_lst_cache.py --exp E3-lsc-v1 --split train
$ pytest -q tests/test_cache.py
```

✔ **Accept when:** cache exists for all train videos; one batch from the loader prints expected shapes; disk size noted.

P21 Train the LST-Matcher

Plan: T5.3 | Env: logo-torch (GPU)

Depends on: P20

You provide (paste with the prompt): Paste the optimizer/scheduler part of GoMatching's training script for reference, and your VRAM use from a smoke run if it fails.

PROMPT (paste after Block A):

```
TASK P21. Write src/logo/tracking/train_lst.py and configs/E5-lst.yaml: train LSTAdapter on LSTCacheDataset.
AdamW lr 5e-5, warm-up then cosine, AMP, clip length 6 (reduce if OOM), iterations from config (smoke mode:
200 iterations), log association loss every 20 iterations, validate every N iterations by running the
tracker on val cache and computing IDF1 through tools/evaluate.py, keep best checkpoint by validation IDF1,
resumable. Report peak VRAM. Keep third_party code unmodified.
```

You run:

```
$ python -m logo.tracking.train_lst --config configs/E5-lst.yaml --smoke
$ python -m logo.tracking.train_lst --config configs/E5-lst.yaml
```

✔ **Accept when:** validation IDF1 beats the P16 baseline. **Cut-line:** if not working by the sprint-5 deadline use fallback P21F.

P21F Fallback tracker: ByteTrack-style + VPMM embedding

Plan: T5.3 fallback | Env: logo-torch

Depends on: P18, P16

You provide (paste with the prompt): Use only if P21 is blocked.

PROMPT (paste after Block A):

```
TASK P21F (fallback). Extend simple_tracker with tracker.type 'embed': train a small projection head (reuse
VPMM output [N,2d] -> 128-d) with a contrastive/triplet loss on LSTCacheDataset pairs (same GT track =
positive, different = negative, within a 6-frame window), then use cosine distance mixed with IoU in the
association cost (cost = (1-lam)*(1-IoU) + lam*cosine, lam from config). Implement train_embed.py, the
tracker option, and tests. Report IDF1 vs the plain IoU tracker.
```

You run:

```
$ python -m logo.tracking.train_embed --config configs/E5-embed.yaml
$ python scripts/run_pipeline.py --config configs/E5-embed.yaml --stage track,eval
```

✔ **Accept when:** IDF1 improves over the P16 tracker or the result is documented; story stays valid because the paper's own ablation uses ByteTrack.

P22 LST inference, thresholds, trajectory filtering

Plan: T5.4 | Env: logo-torch

Depends on: P21 (or P21F)

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P22. Add tracker.type 'lst' to logo.tracking.run_track (uses LSTAdapter.associate over frames online).
Inputs below tracker.thr (default 0.4 for icdar2015v and icdar2013v, 0.3 for dstext) are dropped before
matching. Post-process like the paper: remove trajectories whose mean score is below tracker.traj_thr (0.6)
and, in spotting mode, trajectories whose text is empty or a single character. Write
tools/sweep_threshold.py producing a CSV and PNG of MOTA and IDF1 (tracking and spotting) versus tracker.thr
in {0.3,...,0.9}; it must refuse the test split.
```

You run:

```
$ python scripts/run_pipeline.py --config configs/E5-lst.yaml --stage track,eval
$ python tools/sweep_threshold.py --config configs/E5-lst.yaml --split val
```

✔ **Accept when:** tracking and spotting metrics plus the sweep curve saved.

P23 Baseline freeze and efficiency profile

Plan: T5.5 | Env: logo-torch

Depends on: P22

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P23. Write tools/freeze_baseline.py: (1) run the four ablation configs (Baseline, +LSC hard, +GS,
+VPMM/LST) through run_pipeline on val; (2) profile each stage on 200 frames: parameters,
torch.cuda.max_memory_allocated, ms/frame, end-to-end FPS at batch 1; (3) rerun one config and assert
metrics match the stored ones within 0.3; (4) generate docs/baseline_report.md with the reproduction table,
```

```
a column 'paper reported (Table V)' for comparison, the efficiency table and a list of deviations read from docs/decisions.md; (5) print the git commands to tag v1.0-base. Add tools/profile_efficiency.py that P28 will reuse.
```

You run:

```
$ python tools/freeze_baseline.py --split val
$ git tag v1.0-base
```

✔ **Accept when:** MVP GATE: baseline report exists, reproducible within +-0.3. Reproduction success = same ordering of component gains as the paper, or a documented reason.

Sprint 6: Our contribution

P24 Error analysis on the frozen baseline

Plan: T6.1 | Env: logo-torch

Depends on: P23

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P24. Write tools/error_analysis.py using per-frame matching at IoU 0.5 against GT (dont_care handled as in the evaluator). Categories: missed GT (split by text height bins: below 20 px, 20-50 px, above 50 px), false positives (unmatched predictions), wrong transcription (matched but text differs after normalization), ID switches and fragmentations (from the accumulator). Output errors.csv (counts per category and size bin), a bar-chart PNG, and an image gallery (6 examples per category, cropped frames with GT in green and prediction in red) under docs/figs/errors/. Print the two largest error categories in one sentence.
```

You run:

```
$ python tools/error_analysis.py --exp E5-lst-v1 --split val
```

✔ **Accept when:** taxonomy table and gallery committed; top-2 error types named and used to justify P25 and P26.

P25 C2: soft confidence-aware language re-scoring

Plan: T6.2 | Env: logo-torch

Depends on: P24

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P25. Implement mode 'soft' in logo.scoring.fusion.fuse: lang_soft = 1 - p_eos (probability that the first decoded token is NOT the end token), alternative lang_soft_conf = text_conf (mean character probability) selected by fusion.soft_source in {'p_eos','text_conf'}; fused = (1-w)*det + w*lang_soft. Keep 'hard' unchanged. Write tools/sweep_fusion.py: grid over w in {0.3,0.5,0.7}, mode in {hard,soft}, tracker.thr in {0.4,...,0.9}; runs only fuse,track,eval (LSC outputs already cached) on val; refuses test; writes CSV and two PNG plots (MOTA vs thr, IDF1 vs thr; hard vs soft). Add tests for fuse values.
```

You run:

```
$ pytest -q tests/test_fusion.py
$ python tools/sweep_fusion.py --config configs/E5-lst.yaml --split val
```

✔ **Accept when:** CSV + plots saved; you can state whether soft improves a metric or flattens the MOTA/IDF1 trade-off. Fix w and thr on val BEFORE any final run.

P26 C3: track-level text voting and filtering

Plan: T6.3 | Env: logo-torch

Depends on: P25

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P26. Implement logo.voting.track_vote.vote_tracks(tracks, cfg) (tracks = list of track-jsonl dicts, also needs per-box text_conf and lang score: carry them via extra keys 'text_conf' and 'lang' if present, defaulting to score). For each track_id: normalize texts (lowercase, keep a-z0-9, keep original for output), weight each frame's text by text_conf*lang, pick the string with the highest summed weight (tie -> highest mean confidence), assign it to every box of the track. Track filter: drop tracks whose mean lang is below voting.tau or whose length is below voting.min_len. Wire into run_pipeline stage 'vote' (pass-through if voting.enabled is false), write final tracks to runs/<exp>/vote/. Tool tools/sweep_voting.py over tau in {0.0,0.3,0.5,0.7} and min_len in {1,3,5,10} on val; refuses test. Tests with synthetic tracks: noisy strings around one truth converge to the truth; short low-confidence tracks are removed.
```

You run:

```
$ pytest -q tests/test_voting.py
```

$ `python tools/sweep_voting.py --config configs/E6-vote.yaml --split val`

✔ **Accept when:** spotting MOTA/IDF1 and number of wrong-transcription boxes reported with vs without voting.

P27 Ablation runner and bootstrap confidence intervals

Plan: T6.4 + T6.5 | Env: logo-torch

Depends on: P26

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P27. (1) tools/run_ablation.py reads configs/ablation.yaml (list of named configs: baseline, +LSC hard, +GS, +VPMM/LST = reproduced LOGO, +soft, +voting, all) and runs run_pipeline for each (skipping finished ones), saving runs/<exp>/metrics.json. (2) tools/bootstrap_ci.py: using metrics.json['per_video'] counts, resample videos with replacement 1000 times (seeded) and compute 95% intervals for MOTA = 1 - (misses+fp+switches)/num_objects and IDF1 = 2*idtp/(2*idtp+idfp+idfn) for two runs and for their difference; print a table; flag differences whose interval excludes zero. Tests with synthetic per-video counts (identical runs give a difference interval centered at 0).
```

You run:

$ `python tools/run_ablation.py --config configs/ablation.yaml --split val`

$ `python tools/bootstrap_ci.py --a E5-lst-v1 --b E6-all-v1`

✔ **Accept when:** complete ablation table; CI table saved; the report says 'improves' only where the interval excludes zero.

Sprint 7 and 8: Final runs, demo, docs, paper

P28 Final runs and efficiency table

Plan: T7.1 + T7.2 | Env: logo-torch (+ GPU for DSText if available)

Depends on: P27

You provide (paste with the prompt): Your frozen best config names and chosen w, thr, tau, min_len.

PROMPT (paste after Block A):

```
TASK P28. Write scripts/final_eval.py --config <best> --final: runs the full pipeline on the held-out split (test, or val if test labels are unavailable, as recorded in docs/decisions.md) for icdar2015v and icdar2013v (and dstext if det/lsc/cache exist), saves runs/FINAL-<ds>/metrics.json with a meta block (git commit, config hash, seed, GPU name). Extend tools/profile_efficiency.py to produce runs/FINAL-efficiency.json (params, peak VRAM, ms/frame per stage). Refuse to run without --final and refuse to change any hyperparameter (read them only from the config).
```

You run:

$ `python scripts/final_eval.py --config configs/FINAL.yaml --final`

✔ **Accept when:** FINAL metrics for each dataset; any skipped dataset listed as a limitation.

P29 Tables and figures generated by script

Plan: T7.3 + T7.5 | Env: logo-torch

Depends on: P28

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P29. Write tools/make_tables.py reading runs/*/metrics.json and writing paper/tables/*.tex (booktabs, numbers with 1-2 decimals, best value bold) and docs/tables/*.md for: ablation (S1), comparison (our reproduced rows, plus a separate block of 'reported' published numbers read from configs/reported_numbers.yaml which I will fill), threshold sensitivity (S2), voting (S3), efficiency (S4), error analysis (S5), CI. Write tools/make_figures.py producing vector PDFs in paper/figs: threshold curves hard vs soft, error bars, re-score before/after panel, tracking overlay frames with ids, failure-case montage. Numbers are never typed by hand.
```

You run:

$ `python tools/make_tables.py`

$ `python tools/make_figures.py`

✔ **Accept when:** all tables/figures regenerate with one command and open correctly.

P30 Demo

Plan: T7.4 | Env: logo-torch (+ paddle for Plan A detector)

Depends on: P28

You provide (paste with the prompt): Nothing.

PROMPT (paste after Block A):

```
TASK P30. Write scripts/demo.py --video in.mp4 --out out.mp4 [--config FINAL.yaml]: extract frames to a temp dir, run detector (call the paddle-env script through subprocess with the env path from config if Plan A, else the OBB script), LSC, fuse (best config), tracker, voting, then render an mp4 with quads, track ids and recognized text, plus a printed summary (tracks found, longest track, FPS). Optional --gradio flag launching a minimal upload page. Clean temp files. Document usage in README.
```

You run:

```
$ python scripts/demo.py --video samples/clip.mp4 --out runs/demo.mp4
```

✔ **Accept when:** runs end to end on a video the system has never seen; three backup output clips saved.

P31 README and reproducibility guide

Plan: T8.3 | Env: any

Depends on: P29

You provide (paste with the prompt): Your final results table (paste) and checkpoint links.

PROMPT (paste after Block A):

```
TASK P31. Write README.md: one-paragraph overview, pipeline figure placeholder path, installation (both envs, link to docs/env_setup.md), data preparation (no data in repo; where to get ICDAR2013/2015 Video and DSText from RRC; commands for convert/extract/splits), how to reproduce each table (exact commands), results summary table (use only the numbers I paste), checkpoints links, deviations from the paper (read docs/decisions.md), citation of the base paper and acknowledgements, license note. Do not invent any number or link: leave <LINK> where I have not provided one.
Results + links: <paste>
```

You run:

```
$ # a teammate follows the README from a clean clone
```

✔ **Accept when:** teammate reproduces one table from scratch following only the README.

P32 IEEE paper skeleton and first draft

Plan: T8.4 | Env: Overleaf / any

Depends on: P29

You provide (paste with the prompt): Paste docs/baseline_report.md, the generated tables (.md versions) and your bullet list of contributions.

PROMPT (paste after Block A):

```
TASK P32. Create paper/main.tex using IEEEtran (conference mode) with sections: Abstract (150 to 250 words, one paragraph, no citations), Index Terms, I Introduction, II Related Work, III Method (A overview of reproduced pipeline, B soft language re-scoring, C track-level voting, D implementation under 6 GB), IV Experiments (datasets and metrics, implementation details and deviations, main results, ablation, threshold sensitivity, voting, efficiency, qualitative and failure analysis), V Conclusion and Limitations, Acknowledgment, References via paper/refs.bib. Include tables via \input{tables/...}. RULES: every number must come from the pasted tables or generated tex files; where a result is missing write \textbf{[TODO result]} instead of inventing; write in your own words and do not copy sentences from the base paper; cite arXiv:2405.19194, ICDAR 2013/2015 robust reading, DSText, PP-YOLOE-R, ABINet, GoMatching, DeepSolo, ByteTrack, MOT metrics, TransDETR, CoText; put every reference into refs.bib with the fields you are sure of and mark uncertain fields with a comment. Table captions above, figure captions below, Roman section numbers. Add an 'AI-assisted drafting' sentence to Acknowledgment for me to keep or remove per venue policy.
Pasted: <baseline_report> <tables> <contributions>
```

You run:

```
$ # compile in Overleaf; then you verify every claim and number
```

✔ **Accept when:** draft compiles; no number without a source file; every reference checked against the real paper by you.

P33 Project report and final presentation drafts

Plan: T8.1 + T8.2 | Env: any

Depends on: P29, P32

You provide (paste with the prompt): Paste docs/baseline_report.md, docs/decisions.md and the table .md files; your college report template if any.

PROMPT (paste after Block A):

```
TASK P33. Produce (1) a project report outline with full draft text for: Introduction, Literature survey
(video text spotting: YORO, Free, CoText, TransDETR, GoMatching, LOGO), Problem statement and objectives,
Methodology, Implementation (environment, stage design, deviations), Experiments and results, Error
analysis, Conclusion, Future work. (2) A 12-14 slide deck outline (title, bullets, which figure goes where)
for the final review. Use only numbers from the pasted files; mark gaps with [TODO]. Be concrete about what
we reproduced vs what is new (C2, C3), and include a limitations slide and a 'questions we expect' list with
short answers grounded in our results.
Pasted: <files> <college template>
```

You run:

```
$ # review with your guide
```

✔ **Accept when:** guide review done; every number traces to a run.

5. Definition of 'project built as expected'

- All gates in Section 1 passed, tags v0.3-detector, v1.0-base, v2.0-final in Git.
- `pytest -q` passes on CPU; `python scripts/run_pipeline.py --config configs/FINAL.yaml --final` regenerates every metrics.json.
- Tables and figures are produced by `tools/make_tables.py` and `tools/make_figures.py`; the paper and report contain no hand-typed numbers.
- docs/decisions.md lists every deviation from the base paper (detector size, resolution, Plan B, no pseudo-labels, hardware).
- The demo runs on an unseen video; README lets a teammate reproduce one table from a clean clone.

Bring me the task id and the failing piece when something breaks, or the PROJECT_STATE.md when a sprint ends, and I will help adjust the next prompts to what you actually built.